

Today 1:58 PM

I am a gamer. I'm playing a Pygame marble tilt maze where a round marble rolls around and bounces off rectangular walls. The marble should only bounce when it actually touches a wall. Right now, near a wall's corner, it bounces off empty space before it visually touches the wall. Investigate and enhance collision accuracy so bounces only happen when the round marble actually touches a wall. I have pasted my code for

game_engine.py, marble.py, wa
import pygame
from .marble import Marble

[Claim offer](#)[Show more](#)

Yes. The bug is exactly in `_resolve_wall_collisions()`: `colliderect()` compares the **marble's square bounding box** with the wall rectangle. For a circular marble, that square reaches the wall's corner before the circle does, producing the phantom bounce.

The correct test is **circle-vs-rectangle collision**:

1. Find the point on the wall rectangle closest to the marble's center.
2. Measure the distance from the marble center to that point.
3. A collision exists only when that distance is **less than or equal to the marble radius**.
4. Push the marble out along the actual collision normal and reflect only the velocity component along that normal.

Replace your `_resolve_wall_collisions()` with this:

```
def _resolve_wall_collisions(self):
```

Python



Run

```
    # Marble center  
    cx = self.marble.x  
    cy = self.marble.y  
    r = self.marble.radius
```

```
    # Find the closest point on the rectangle to the marble center
```

```

closest_x = max(wall_rect.left, min(cx, wall_rect.right))
closest_y = max(wall_rect.top, min(cy, wall_rect.bottom))

dx = cx - closest_x
dy = cy - closest_y

distance_squared = dx * dx + dy * dy

# No collision: the actual circular marble does not touch the wall
if distance_squared > r * r:
    continue

# -----
# Determine collision normal
# -----

if distance_squared > 0:
    # Normal from wall toward marble
    distance = distance_squared ** 0.5
    nx = dx / distance
    ny = dy / distance
else:
    # Marble center is inside the wall rectangle.
    # Find the nearest wall face and push the marble out.
    distances = [
        (abs(cx - wall_rect.left), -1, 0),
        (abs(wall_rect.right - cx), 1, 0),
        (abs(cy - wall_rect.top), 0, -1),
        (abs(wall_rect.bottom - cy), 0, 1),
    ]

    _, nx, ny = min(distances, key=lambda item: item[0])

# -----
# Push marble out of the wall
# -----

if distance_squared > 0:
    penetration = r - distance

    if penetration > 0:
        self.marble.x += nx * penetration
        self.marble.y += ny * penetration
    else:
        # Center was inside the wall.
        # Move it to the nearest valid position.
        if nx != 0:
            self.marble.x = (
                wall_rect.left - r
                if nx < 0
                else wall_rect.right + r
            )

```

```

else:
    self.marble.y = (
        wall_rect.top - r
        if ny < 0
        else wall_rect.bottom + r
    )

# -----
# Bounce only if moving INTO the wall
# -----

velocity_into_wall = (
    self.marble.vx * nx +
    self.marble.vy * ny
)

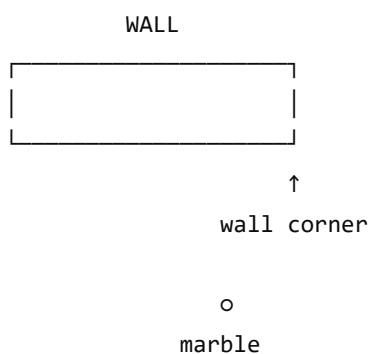
if velocity_into_wall < 0:
    # Reflect the velocity along the collision normal.
    bounce = 0.3

    self.marble.vx -= (
        (1 + bounce) * velocity_into_wall * nx
    )
    self.marble.vy -= (
        (1 + bounce) * velocity_into_wall * ny
    )

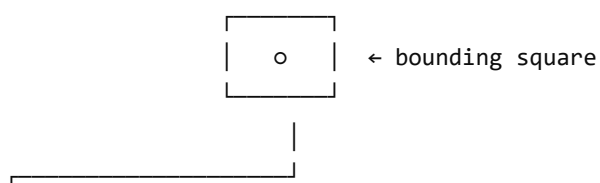
```

Why this fixes your corner problem

Suppose the wall ends here:



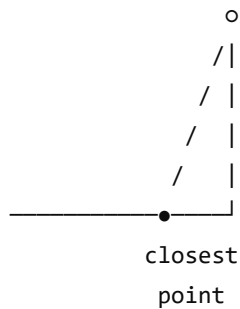
With your current code, the marble's invisible square bounding box might overlap that corner:





`colliderect()` sees an overlap and bounces the marble, even though the **circle itself hasn't touched the corner**.

The new algorithm instead calculates:



It checks:

</> Python



▶ Run

```
distance(center, closest_point) <= radius
```

So the marble only collides when its **actual circular edge** reaches the wall.

One additional important improvement

Your old code always did:

</> Python



▶ Run

```
self.marble.vx *= -0.3
```

or

</> Python



▶ Run

```
self.marble.vy *= -0.3
```

That assumes every collision is either perfectly horizontal or perfectly vertical.

That's not correct at a corner. The new version uses the collision normal:

</> Python



▶ Run

```
velocity_into_wall = self.marble.vx * nx + self.marble.vy * ny
```

and reflects the velocity along that normal. This means corner collisions behave naturally instead of arbitrarily flipping only `vx` or `vy`.

You don't need to change `marble.py` or `wall.py`

Your existing classes are fine:

`</> Python`



`▶ Run`

```
class Marble:
    ...
```

and

`</> Python`



`▶ Run`

```
class Wall:
    ...
```

The main problem is entirely in the collision detection/resolution in `game_engine.py`.

One thing to be aware of: this is a **discrete collision test**—the marble moves first and is then corrected. At very high speeds, a marble could theoretically move through a thin wall between frames. Your `max_speed = 9` and wall thickness of `16` make that less likely, but if you later see tunneling, the next improvement would be **continuous/swept circle collision detection**.



I am a gamer. I'm playing the same Pygame marble tilt maze, and the collision fix from before is working. When a round ends, I want an on-screen result and a way to play again, instead of a line in the console. Add a screen that displays whether the maze was solved (with the finish time) or the timer ran out, then gracefully waits for input instead of just printing to the console.

You only need to change `game_engine.py`. The cleanest approach is to make the game enter a **game-over state**, render a result overlay, and restart when the player presses a key.

1. Replace `handle_event()`

</> Python



Run

```
def handle_event(self, event):
    if not self.game_over:
        return

    if event.type == pygame.KEYDOWN:
        if event.key in (pygame.K_r, pygame.K_RETURN, pygame.K_SPACE):
            self._restart_game()
        elif event.key == pygame.K_ESCAPE:
            pygame.quit()
            raise SystemExit
```

2. Add a restart method

Put this inside `GameEngine` :

</> Python



Run

```
def _restart_game(self):
    self.marble = Marble(50, 50)

    self.start_ticks = pygame.time.get_ticks()

    self.game_over = False
    self.result = None
    self.finish_time_ms = None

    # Allow the end-screen message to appear again later
    self._game_over_logged = False
```

3. Change the end of `update()`

You can remove the old console-oriented behavior. Your `update()` can remain mostly the same, but make sure the timeout section is:

</> Python



Run

```
if elapsed >= self.time_limit_ms:
    self.game_over = True
    self.result = "timeout"
    self.finish_time_ms = None
    return
```

And your goal section can remain:

</> Python



▶ Run

```

if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
    self.game_over = True
    self.result = "solved"
    self.finish_time_ms = elapsed

```

4. Replace the end-screen part of render()

Remove this old section:

</> Python



▶ Run

```

if self.game_over and not getattr(self, "_game_over_logged", False):
    if self.result == "solved":
        print(f"Solved! Finished in {self.finish_time_ms / 1000:.1f}s")
    else:
        print("Time's up! Maze not solved.")
    self._game_over_logged = True

```

and replace it with:

</> Python



▶ Run

```

if self.game_over:
    # Dark translucent overlay
    overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 180))
    screen.blit(overlay, (0, 0))

    # Fonts
    title_font = pygame.font.SysFont("Arial", 48, bold=True)
    message_font = pygame.font.SysFont("Arial", 30)
    instruction_font = pygame.font.SysFont("Arial", 24)

    if self.result == "solved":
        title_text = "Maze Solved!"
        message_text = (
            f"Finished in {self.finish_time_ms / 1000:.1f} seconds"
        )
    else:
        title_text = "Time's Up!"
        message_text = "The maze was not solved."

    title_surface = title_font.render(title_text, True, WHITE)
    message_surface = message_font.render(message_text, True, WHITE)
    instruction_surface = instruction_font.render(
        "Press R, Enter, or Space to play again",
        True,
        WHITE

```

```

    )

    # Center everything on screen
    title_rect = title_surface.get_rect(
        center=(self.width // 2, self.height // 2 - 70)
    )

    message_rect = message_surface.get_rect(
        center=(self.width // 2, self.height // 2)
    )

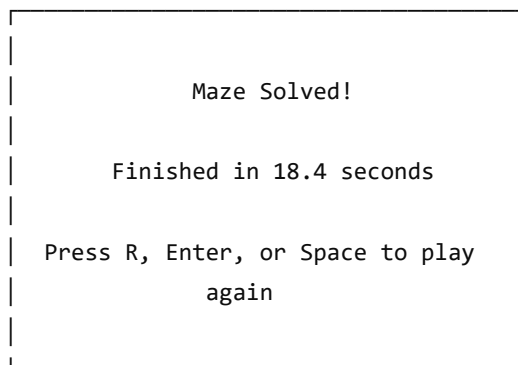
    instruction_rect = instruction_surface.get_rect(
        center=(self.width // 2, self.height // 2 + 60)
    )

    screen.blit(title_surface, title_rect)
    screen.blit(message_surface, message_rect)
    screen.blit(instruction_surface, instruction_rect)

```

Result

When the player wins:



```

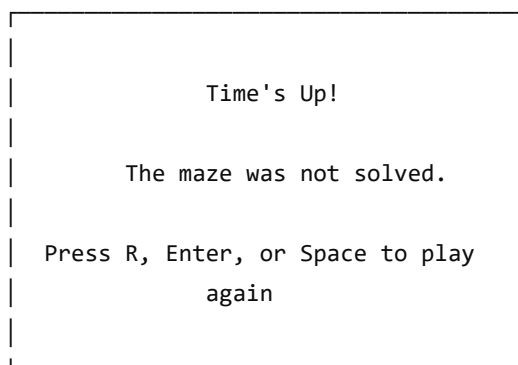
Maze Solved!

Finished in 18.4 seconds

Press R, Enter, or Space to play
again

```

When the timer expires:



```

Time's Up!

The maze was not solved.

Press R, Enter, or Space to play
again

```

The important part is that `update()` already stops doing game logic when `self.game_over` is `True`:

</> Python



▶ Run

```
if self.game_over:
    return
```

So the marble freezes on the final state, the result screen stays visible, and the game waits for input rather than repeatedly printing messages to the console.



I am a gamer. I'm playing the same Pygame marble tilt maze

After the end screen, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard tilt strength/friction/time limit), or exit.

Yes. Since you already have the **end screen**, the clean way is to add a **difficulty-selection screen after the game ends**.

The flow becomes:

Playing → Solved/Time's Up → Difficulty screen → Easy / Medium / Hard / Exit → New game

You can keep `marble.py` and `wall.py` unchanged. Replace/update `game_engine.py` as follows.

1. Add difficulty settings in `__init__`

Replace your current:

</> Python



▶ Run

```
self.tilt_strength = 0.6
self.friction = 0.02
self.max_speed = 9
```

with:

</> Python



▶ Run

```
self.difficulty = None

self.difficulty_settings = {
```

```
    "Easy": {
        "tilt_strength": 0.4,
        "friction": 0.04,
        "time_limit_ms": 60000
    },
    "Medium": {
        "tilt_strength": 0.6,
        "friction": 0.02,
        "time_limit_ms": 45000
    },
    "Hard": {
        "tilt_strength": 0.9,
        "friction": 0.01,
        "time_limit_ms": 30000
    }
}

self.tilt_strength = 0.6
self.friction = 0.02
self.max_speed = 9
```

Then change:

</> Python



▶ Run

```
self.time_limit_ms = 45000
```

to:

</> Python



▶ Run

```
self.time_limit_ms = 45000
```

You can leave that because Medium is the default, but the difficulty screen will set it when a new game starts.

2. Add a difficulty-selection state

In `__init__`, add:

</> Python



▶ Run

```
self.show_difficulty_screen = False
```

So you now have:

</> Python



▶ Run

```
self.game_over = False
self.result = None
self.finish_time_ms = None
self.show_difficulty_screen = False
```

3. Replace `handle_event()`

Use this complete version:

</> Python



▶ Run

```
def handle_event(self, event):
    if event.type != pygame.KEYDOWN:
        return

    # Difficulty selection screen
    if self.show_difficulty_screen:
        if event.key == pygame.K_1:
            self._start_new_game("Easy")

        elif event.key == pygame.K_2:
            self._start_new_game("Medium")

        elif event.key == pygame.K_3:
            self._start_new_game("Hard")

        elif event.key == pygame.K_ESCAPE:
            pygame.quit()
            raise SystemExit

    return

# End screen
if self.game_over:
    if event.key in (
        pygame.K_r,
        pygame.K_RETURN,
        pygame.K_SPACE
    ):
        self.show_difficulty_screen = True

    elif event.key == pygame.K_ESCAPE:
        pygame.quit()
        raise SystemExit
```

Now pressing **R / Enter / Space** on the result screen takes the player to difficulty selection instead of immediately restarting.

4. Add `_start_new_game()`

Add this method inside `GameEngine` :

</> Python



▶ Run

```
def _start_new_game(self, difficulty):
    settings = self.difficulty_settings[difficulty]

    self.difficulty = difficulty

    self.tilt_strength = settings["tilt_strength"]
    self.friction = settings["friction"]
    self.time_limit_ms = settings["time_limit_ms"]

    # Reset marble
    self.marble = Marble(50, 50)

    # Reset game state
    self.game_over = False
    self.result = None
    self.finish_time_ms = None

    # Hide difficulty screen
    self.show_difficulty_screen = False

    # Restart timer
    self.start_ticks = pygame.time.get_ticks()
```

5. Change the beginning of `update()`

At the beginning of `update()` , use:

</> Python



▶ Run

```
def update(self):
    if self.game_over or self.show_difficulty_screen:
        return
```

This is important because the marble should **not move** while either screen is being displayed.

6. Replace your `render()` with this version

This keeps your maze and timer rendering, then adds the appropriate screen on top.

`</>` Python



▶ Run

```
def render(self, screen):
    screen.fill(DARK)

    # Draw maze
    for wall in self.walls:
        pygame.draw.rect(screen, WALL_COLOR, wall.rect())

    # Draw goal
    pygame.draw.circle(
        screen,
        GOAL_COLOR,
        (self.goal_x, self.goal_y),
        self.goal_radius
    )

    # Draw marble
    pygame.draw.circle(
        screen,
        WHITE,
        (int(self.marble.x), int(self.marble.y)),
        self.marble.radius
    )

    # Draw timer only while playing
    if not self.game_over and not self.show_difficulty_screen:
        elapsed = pygame.time.get_ticks() - self.start_ticks
        seconds_left = max(
            0,
            (self.time_limit_ms - elapsed) // 1000
        )

        timer_text = self.font.render(
            f"Time: {seconds_left}s",
            True,
            WHITE
        )

        screen.blit(timer_text, (10, 10))

    # Show current difficulty
    difficulty_text = self.font.render(
        f"Difficulty: {self.difficulty or 'Medium'}",
        True,
        WHITE
    )
```

```
        screen.blit(
            difficulty_text,
            (10, 45)
        )

    # Difficulty selection screen
    if self.show_difficulty_screen:
        self._render_difficulty_screen(screen)

    # Game-over screen
    elif self.game_over:
        self._render_game_over_screen(screen)
```

7. Add the game-over screen method

Add:

</> Python



▶ Run

```
def _render_game_over_screen(self, screen):
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )
    overlay.fill((0, 0, 0, 190))
    screen.blit(overlay, (0, 0))

    title_font = pygame.font.SysFont(
        "Arial",
        48,
        bold=True
    )

    message_font = pygame.font.SysFont(
        "Arial",
        30
    )

    instruction_font = pygame.font.SysFont(
        "Arial",
        24
    )

    if self.result == "solved":
        title_text = "Maze Solved!"
        message_text = (
            f"Finished in "
            f"{self.finish_time_ms / 1000:.1f} seconds"
        )
```

```
else:
    title_text = "Time's Up!"
    message_text = "The maze was not solved."

title_surface = title_font.render(
    title_text,
    True,
    WHITE
)

message_surface = message_font.render(
    message_text,
    True,
    WHITE
)

instruction_surface = instruction_font.render(
    "Press R, Enter, or Space to continue",
    True,
    WHITE
)

screen.blit(
    title_surface,
    title_surface.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 70
        )
    )
)

screen.blit(
    message_surface,
    message_surface.get_rect(
        center=(
            self.width // 2,
            self.height // 2
        )
    )
)

screen.blit(
    instruction_surface,
    instruction_surface.get_rect(
        center=(
            self.width // 2,
            self.height // 2 + 60
        )
    )
)
```

8. Add the difficulty-selection screen

Finally, add:

</> Python



▶ Run

```
def _render_difficulty_screen(self, screen):
    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA
    )
    overlay.fill((0, 0, 0, 210))
    screen.blit(overlay, (0, 0))

    title_font = pygame.font.SysFont(
        "Arial",
        44,
        bold=True
    )

    option_font = pygame.font.SysFont(
        "Arial",
        30
    )

    instruction_font = pygame.font.SysFont(
        "Arial",
        22
    )

    title_surface = title_font.render(
        "Choose Difficulty",
        True,
        WHITE
    )

    screen.blit(
        title_surface,
        title_surface.get_rect(
            center=(
                self.width // 2,
                self.height // 2 - 120
            )
        )
    )

    options = [
        ("1", "Easy", "60 seconds"),
        ("2", "Medium", "45 seconds"),
        ("3", "Hard", "30 seconds"),
```

```

]

start_y = self.height // 2 - 50

for i, (key, name, time_limit) in enumerate(options):
    text = f"{key} - {name} ({time_limit})"

    surface = option_font.render(
        text,
        True,
        WHITE
    )

    screen.blit(
        surface,
        surface.get_rect(
            center=(
                self.width // 2,
                start_y + i * 55
            )
        )
    )

instruction = instruction_font.render(
    "Press 1, 2, or 3 to start | Esc to exit",
    True,
    WHITE
)

screen.blit(
    instruction,
    instruction.get_rect(
        center=(
            self.width // 2,
            self.height // 2 + 130
        )
    )
)
)

```

Your resulting flow

After completing the maze:

Maze Solved!



Finished in 23.7 seconds

Press R, Enter, or Space
to continue

Press R :

Choose Difficulty

1 - Easy (60 seconds)

2 - Medium (45 seconds)

3 - Hard (30 seconds)

Press 1, 2, or 3 to start | Esc to exit

Then:

- 1 → **Easy**: lower tilt strength, more friction, 60 seconds
- 2 → **Medium**: normal settings, 45 seconds
- 3 → **Hard**: stronger tilt, less friction, 30 seconds
- **Esc** → **Exit**

The important improvement is that **difficulty selection happens only after the round ends**, so the existing gameplay and your corrected circle-vs-rectangle collision system don't need to be changed.



I am a gamer. I'm playing the finished Pygame marble tilt maze with all features in.
Add basic sound effects for bouncing off a wall, reaching the goal, and the timer running out.

Yes. Since your maze is already finished, the simplest approach is to use Pygame's built-in mixer and generate short sound effects in code, so you **don't need to download .wav files**.

You only need to modify `game_engine.py` .

1. Initialize the sound system

At the beginning of `__init__` , after `pygame` is available, add:

</> Python

```
pygame.mixer.init()
```

Run

Then add:

</> Python



▶ Run

```
self._load_sounds()
```

So the relevant part becomes:

</> Python



▶ Run

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        pygame.mixer.init()
        self._load_sounds()

        self.marble = Marble(50, 50)

        # ... rest of your existing code
```

2. Add a sound-loading method

Add this inside GameEngine :

</> Python



▶ Run

```
def _load_sounds(self):
    """
    Create simple sound effects programmatically.
    No external sound files are required.
    """

    sample_rate = 44100

    def make_tone(frequency, duration, volume=0.3):
        import math
        import array

        samples = int(sample_rate * duration)
        buffer = array.array("h")

        for i in range(samples):
            t = i / sample_rate

            # Fade out toward the end to avoid clicks
            fade = 1.0 - (i / samples)
```

```

        value = (
            math.sin(2 * math.pi * frequency * t)
            * volume
            * fade
        )

        buffer.append(int(value * 32767))

    return pygame.mixer.Sound(buffer=buffer)

# Wall bounce: short, low "thump"
self.bounce_sound = make_tone(
    180,
    0.08,
    0.35
)

# Goal: higher, Longer tone
self.goal_sound = make_tone(
    700,
    0.30,
    0.35
)

# Timeout: Lower tone
self.timeout_sound = make_tone(
    120,
    0.45,
    0.35
)

```

This creates three sounds:

-  **Bounce:** short low sound
-  **Goal:** higher sound
-  **Timeout:** low sound

3. Play the bounce sound

In your `_resolve_wall_collisions()` , after you've confirmed an actual collision, play:

`</>` Python



 Run

```
self.bounce_sound.play()
```

However, don't put it somewhere that can trigger repeatedly while the marble is sitting against the wall.

A good place is immediately when you detect that the marble is moving **into** the wall:

</> Python



▶ Run

```
velocity_into_wall = (
    self.marble.vx * nx +
    self.marble.vy * ny
)

if velocity_into_wall < 0:
    self.bounce_sound.play()

    bounce = 0.3

    self.marble.vx -= (
        (1 + bounce) * velocity_into_wall * nx
    )

    self.marble.vy -= (
        (1 + bounce) * velocity_into_wall * ny
    )
```

Because your collision system already checks the **actual circle-to-rectangle distance**, the sound will correspond to a genuine wall collision rather than the old phantom corner collision.

4. Play the goal sound

In `update()` , you currently have something like:

</> Python



▶ Run

```
if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
    self.game_over = True
    self.result = "solved"
    self.finish_time_ms = elapsed
```

Change it to:

</> Python



▶ Run

```
if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
    self.game_over = True
    self.result = "solved"
    self.finish_time_ms = elapsed

    self.goal_sound.play()
```

Now the goal sound plays exactly once when the marble reaches the goal.

5. Play the timeout sound

Your timeout section should be:

</> Python



▶ Run

```
if elapsed >= self.time_limit_ms:
    self.game_over = True
    self.result = "timeout"
    self.finish_time_ms = None

    self.timeout_sound.play()

return
```

So when the timer reaches zero, the timeout sound plays and your existing result screen appears.

6. Important: avoid repeated bounce sounds

Because your game runs `update()` many times per second, it's possible for the marble to remain in contact with a wall for multiple frames.

The velocity check:

</> Python



▶ Run

```
if velocity_into_wall < 0:
```

helps prevent this because after the bounce the velocity is directed away from the wall.

If you find that the sound still triggers too rapidly when sliding along a wall, add a small cooldown.

In `__init__`:

</> Python



▶ Run

```
self.last_bounce_sound = 0
self.bounce_sound_cooldown = 80
```

Then replace:

</> Python



▶ Run

```
self.bounce_sound.play()
```

with:

</> Python



▶ Run

```
now = pygame.time.get_ticks()

if now - self.last_bounce_sound >= self.bounce_sound_cooldown:
    self.bounce_sound.play()
    self.last_bounce_sound = now
```

I'd recommend this version for your game because the marble can potentially make several collision checks in quick succession.

Final behavior

Your completed game will now have:

Wall collision

→ short *thump*

Reach goal

→ higher *ding*

Timer reaches zero

→ low *timeout* sound

Solved/timeout

→ existing result screen

Play again

→ difficulty selection

And because the sounds are generated programmatically, there are **no extra audio files or asset folders to manage**.



ChatGPT can make mistakes. Check important info.